

Why Joins Exist

15.1.1 Why Relational Databases Exist

Imagine you're building a college management system for LNMIIT.

Students:

Student_id	Name
1	Arpit
2	Aadz

Courses

Course_id	Course_name
101	DBMS
102	Operating Systems

Enrollments

Student_id	Course_id
1	101
1	102
2	101

Notice something:

We never repeated

Arpit
DBMS

multiple times.

Instead, each fact is stored once. This is called Normalization.

15.1.2 Why Split Data?

Suppose we stored everything in one table.

Student	Course	Teacher
Arpit	DBMS	Dr Sharma
Arpit	OS	Dr Verma
Aadz	DBMS	Dr Sharma

Problems:

If Dr Sharma changes name



Update many rows.

If DBMS deleted accidentally



Teacher information lost.

Lots of duplication.

Instead we split into

- Students
- Courses
- Enrollments
- Teachers
- Departments

Every table stores one concept.

15.1.3 Primary Key Recap

Every table needs a unique identifier.

Students

Student_id	Name
1	Arpit
2	Aadz

Primary Key

student_id

Uniquely identifies every student.

15.1.4 Foreign Key Recap

Enrollment

Student_id	Course_id
1	101

student_id points to Students.student_id

course_id points to Courses.course_id

Foreign keys connect tables.

15.1.5 Why Can't We Keep Everything Together?

Suppose Aadz changes email.

Without normalization

Need to update 30 rows.

Miss one row



Database becomes inconsistent.

Normalization avoids this.

15.1.6 Relationship Types

There are four major relationships.

One-to-One

Example: Passport ↔ Person

One person



One passport

One passport



One person



Example

Users

Id	Name
1	Arpit

Passport

Passport	User_id
P101	1

One-to-Many

Most common.

Department

Dept_id	Name
1	ECE

Employees

Id	Dept_id
1	1
2	1
3	1

One department



Many employees



Many-to-One

Simply reverse of previous.

Many employees



One department

Many-to-Many

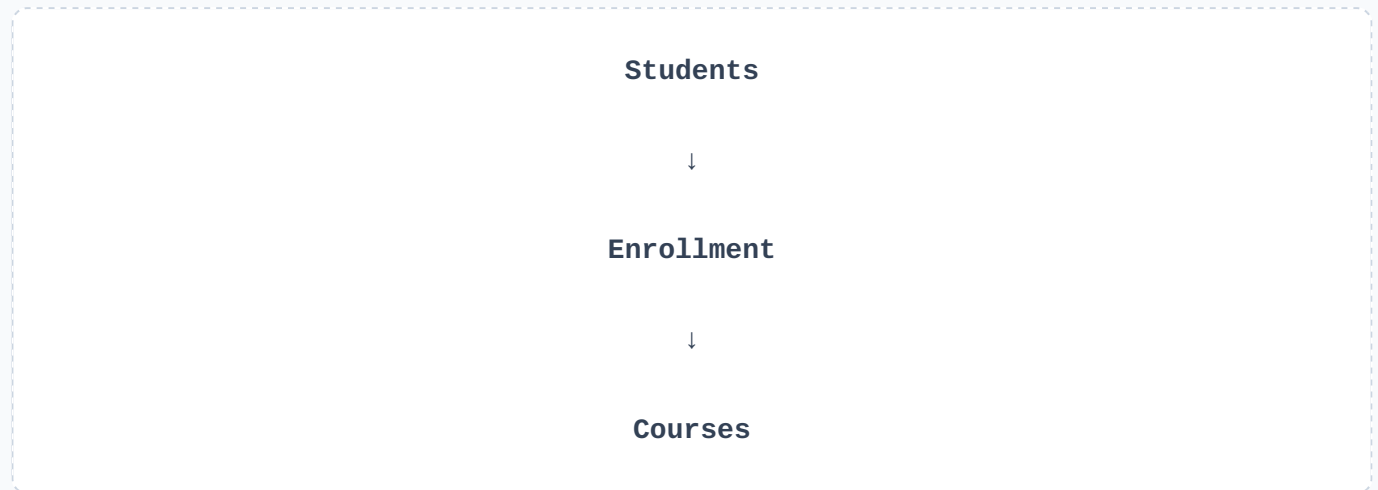
Extremely important.

Students ↓ Courses

One student takes many courses.

One course has many students.

Need bridge table.



Without bridge table Impossible.

15.1.7 Why Duplicate Data Is Bad

Suppose Arpit's department stored 500 times.

Department renamed.

Need 500 updates.

Miss one update.

Database inconsistent.

Normalization avoids this.

15.1.8 What Problem Does JOIN Solve?

Students

Id	Name
1	Arpit

Enrollment

Student_id	Course
1	DBMS

Need output

Name	Course
Arpit	DBMS

Neither table alone has complete information.
JOIN combines them.

Basic JOIN Syntax

```
SELECT columns  
FROM table1  
JOIN table2  
ON condition;
```

Example

```
SELECT *  
FROM Students  
JOIN Enrollment  
ON Students.student_id = Enrollment.student_id;
```

15.2.2 ON Clause

Most important part.

```
ON Students.student_id = Enrollment.student_id
```

Means Connect rows whose IDs are equal.

Without ON SQL doesn't know how tables relate.

USING Clause

Instead of

```
ON Students.student_id = Enrollment.student_id
```

can write

```
USING(student_id)
```

Only works if both columns have same name.
Equivalent.

| Table Aliases

Instead of

```
Students
```

Use

```
Students s
```

Then

```
SELECT s.name  
FROM Students s
```

Cleaner.
Industry standard.

Read first table



Read second table



Find matching rows



Return matching pairs

ON determines matching rows during the join.

WHERE filters rows after the join result is formed.

INNER JOIN returns only rows that satisfy the join condition in both tables.

No match ↓ No output.

Suppose **Students**

Id	Name
1	Arpit
2	Aadz
3	Rahul

Enrollment

Student_id	Course
1	DBMS
1	OS
2	Networks

Query

```
SELECT
  s.name,
  e.course
FROM Students s
INNER JOIN Enrollment e
ON s.id=e.student_id;
```

Output

Student	Course
Arpit	DBMS
Arpit	OS
Aadz	Networks

Indexes on join columns dramatically improve joins.

Classic many-to-many relationship.

```
SELECT
  s.name,
  c.course_name
FROM Students s
JOIN Enrollment e
ON s.id=e.student_id
JOIN Courses c
ON e.course_id=c.course_id;
```

Can INNER JOIN return duplicates?

Yes. One row is returned for every successful match. If one parent row matches multiple child rows, it appears multiple times.

Does INNER JOIN return NULL rows?

No. Rows without a match are excluded.

LeetCode 1068 — Product Sales Analysis I ★ (Warm-up)

Concepts

- Basic INNER JOIN
- Foreign Key
- Selecting columns from multiple tables

Tables

Sales

Sale_id	Product_id	Year	Quantity	Price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product

Product_id	Product_name
100	Nokia
200	Apple

Need

|product_name|year|price|

Thinking

Sales table doesn't contain product name.

Product table doesn't contain year.

Need both.



Join on `product_id`

Solution

```
SELECT
    p.product_name,
    s.year,
    s.price
FROM Sales s
INNER JOIN Product p
ON s.product_id = p.product_id;
```

LeetCode 1378 — Replace Employee ID With The Unique Identifier ★

Concepts

- Simple INNER JOIN
- Aliases

EmployeeUNI

Id	Unique_id
1	11
3	13

Employees

Id	Name
1	Arpit
2	Aadz
3	Rahul

Need

|unique_id|name|

Thinking

Need Employee name, Unique ID.

Neither table has both.

Join.

Solution

```
SELECT
    eu.unique_id,
    e.name
FROM Employees e
INNER JOIN EmployeeUNI eu
ON e.id = eu.id;
```

LeetCode 197 — Rising Temperature ★★

Concepts

- Self INNER JOIN
- Date arithmetic

Weather

Id	RecordDate	Temperature
1	2024-01-01	10
2	2024-01-02	25
3	2024-01-03	20

Need ids where today's temperature > yesterday's.

Thinking

Need to compare today row with yesterday row.

A table joins with itself.

Imagine

Today



Weather

Yesterday



Weather

Join condition

```
today.date = yesterday.date+1
```

Solution (MySQL)

```
SELECT
    w1.id
FROM Weather w1
INNER JOIN Weather w2
ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
WHERE w1.temperature > w2.temperature;
```

LeetCode 1661 — Average Time of Process per Machine ★★

Concepts

- Self Join
- Aggregation
- AVG()

Activity

Machine_id	Process_id	Activity_type	Timestamp
0	0	start	0.700
0	0	end	1.5

Need Average `end-start` per machine.

Thinking

Need Start row, End row of same process.

Join table with itself.

Conditions:

Same machine

Same process

One row=start

Other=end

Solution

```

SELECT
    a.machine_id,
    ROUND(AVG(b.timestamp-a.timestamp),3) AS processing_time
FROM Activity a
INNER JOIN Activity b
ON a.machine_id=b.machine_id
AND a.process_id=b.process_id
WHERE
a.activity_type='start'
AND
b.activity_type='end'
GROUP BY a.machine_id;

```

Students and Examinations ★★

Although this problem ultimately requires a CROSS JOIN, it also relies on an INNER JOIN to count matching exam records, making it a good bridge to the next topic.

Students

Student_id	Student_name
1	Arpit
2	Aadz

Subjects

Subject_name
Math
DBMS

Examinations

Student_id	Subject_name
1	Math
1	Math
2	DBMS

Need Every student, Every subject, Attendance count.

Thinking

Need all possible student-subject pairs first (using CROSS JOIN), then count matching examination records with a join.

Solution

```
SELECT
    s.student_id,
    s.student_name,
    sub.subject_name,
    COUNT(e.subject_name) AS attended_exams
FROM Students s
CROSS JOIN Subjects sub
LEFT JOIN Examinations e
ON s.student_id = e.student_id
AND sub.subject_name = e.subject_name
GROUP BY
    s.student_id,
    s.student_name,
    sub.subject_name
ORDER BY
    s.student_id,
    sub.subject_name;
```